

Problem Statement Cryptocurrency Volatility Prediction Java + DSA Pwskills

Cryptocurrency markets are highly volatile, and understanding and forecasting this volatility is crucial for market participants. Volatility refers to the degree of variation in the price of a cryptocurrency over time, and high volatility can lead to significant risks for traders and investors. Accurate volatility prediction helps in risk management, portfolio allocation, and developing trading strategies. In this project, you are required to build a machine learning model to predict cryptocurrency volatility levels based on historical market data such as OHLC (Open, High, Low, Close) prices, trading volume, and market capitalization. The objective is to anticipate periods of heightened volatility, enabling traders and financial institutions to manage risks and make informed decisions. Your final model should provide insights into market stability by forecasting volatility variations, allowing stakeholders to proactively respond to changing market conditions.

Dataset Information You will use a dataset that includes historical daily cryptocurrency price, volume, and market capitalization data for multiple cryptocurrencies. **Dataset:** The dataset consists of daily records for over 50 cryptocurrencies, including features such as date, symbol, open, high, low, close, volume, and market cap.

Data Preprocessing Require Handle missing values and ensure data consistency. Normalize and scale numerical features. Engineer new features related to volatility and liquidity trends.

Project Development Steps

- Data Collection:** Gather historical OHLC, volume, and market cap data from the provided dataset.
- Data Preprocessing:** Handle missing values, clean data, and normalize numerical features.
- Exploratory Data Analysis (EDA):** Analyze data patterns, trends, and correlation.
- Feature Engineering:** Create relevant features such as moving averages, rolling volatility, liquidity ratios (e.g., volume/market cap), and technical indicators (e.g., Bollinger Bands, ATRP).
- Model Selection:** Choose appropriate machine learning models such as time-series forecasting, regression, or deep learning approaches.
- Model Training:** Train the selected model using the processed dataset.
- Model Evaluation:** Assess model performance using metrics such as RMSE, MAE, and R^2 score.

Cryptocurrency Historical Prices Dataset Problem Statement Java + DSA Pwskills

Model Optimization and Deployment

- Hyperparameter Tuning:** Optimize model parameters for better accuracy.
- Model Testing & Validation:** Test the model on unseen data and analyze prediction.
- Local Deployment:** Deploy the trained model locally using Flask or Streamlit for testing.

Expected Deliverables

1. Machine Learning Model: A trained model that predicts cryptocurrency volatility.
2. Data Processing & Evaluation metrics showing how well the model performs.

Feature Engineering
3. Cleaned and prepared dataset
4. A brief explanation of new features added
5. Exploratory Data Analysis (EDA) Report
6. Summary of dataset statistics
7. Basic visualizations (trends, correlations, distributions)
8. Project Documentation
9. High-Level Design (HLD) Document: Overview of system and architecture
10. Low-Level Design (LLD) Document: Breakdown of how each component is implemented
11. Pipeline Architecture: Explanation of data flow from preprocessing to prediction
12. Final Report: A simple summary of findings, model performance, and key insights
Guidelines & Submission Requirement)
13. Code Documentation: Ensure all scripts are well-commented and easy to follow
14. Report Structure: The report must be structured and should clearly explain the methodology followed
15. Diagrams & Visuals: Use appropriate diagrams and plots to explain data processing, model selection, and performance evaluation
16. Deployment: If possible, deploy the model using a simple interface (e.g., Streamlit or Flask API) for testing predictions
Submission Format
The project must be submitted as a GitHub repository or a zipped folder containing:
17. Source Code
18. EDA Report
19. HLD & LLD Document
20. Pipeline Architecture and Documentation
21. Final Report .

Ans :- **1. Dataset**

Use the **Cryptocurrency Historical Prices Dataset** containing:

- Date
- Symbol
- Open
- High
- Low
- Close
- Volume
- Market Cap

2. Data Preprocessing

Perform the following:

- Remove duplicate rows
- Handle missing values

- Convert Date column to datetime
- Sort data by date
- Normalize numerical columns using StandardScaler
- Remove outliers (optional)

Example:

```
df['Date'] = pd.to_datetime(df['Date'])
```

```
df = df.sort_values('Date')
```

```
df.fillna(method='ffill', inplace=True)
```

3. Exploratory Data Analysis

Plot Closing Price

```
plt.figure(figsize=(12,5))  
plt.plot(df['Date'], df['Close'])  
plt.title("Closing Price")  
plt.show()
```

Correlation Heatmap

```
sns.heatmap(df.corr(numeric_only=True),  
annot=True,  
cmap='coolwarm')
```

Distribution

```
sns.histplot(df['Close'], bins=40)
```

4. Feature Engineering

Create useful features.

Daily Return

```
df['Return'] = df['Close'].pct_change()
```

Rolling Volatility

```
df['Volatility'] = df['Return'].rolling(7).std()
```

Moving Average

```
df['MA7'] = df['Close'].rolling(7).mean()
```

```
df['MA30'] = df['Close'].rolling(30).mean()
```

Liquidity Ratio

```
df['Liquidity'] = df['Volume'] / df['Market Cap']
```

Price Range

```
df['Price_Range'] = df['High'] - df['Low']
```

5. Define Target Variable

Predict next-day volatility.

```
df['Target'] = df['Volatility'].shift(-1)
```

```
df.dropna(inplace=True)
```

6. Feature Selection

```
features = [  
    'Open',  
    'High',  
    'Low',  
    'Close',  
    'Volume',  
    'Market Cap',  
    'Return',  
    'MA7',  
    'MA30',  
    'Liquidity',  
    'Price_Range'
```

```
]
```

```
X = df[features]
```

```
y = df['Target']
```

7. Train-Test Split

```
from sklearn.model_selection import train_test_split
```

```
X_train,X_test,y_train,y_test=train_test_split(  
X,  
y,  
test_size=0.2,  
shuffle=False  
)
```

8. Model Selection

Recommended models:

- Linear Regression
- Random Forest Regressor 
- XGBoost Regressor
- LightGBM
- LSTM (Advanced)

A Random Forest is a strong baseline for this project.

```
from sklearn.ensemble import RandomForestRegressor
```

```
model = RandomForestRegressor(  
n_estimators=200,  
random_state=42  
)
```

```
model.fit(X_train,y_train)
```

9. Prediction

```
pred = model.predict(X_test)
```

10. Model Evaluation

```
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
```

```
mae = mean_absolute_error(y_test,pred)
```

```
rmse = np.sqrt(mean_squared_error(y_test,pred))
```

```
r2 = r2_score(y_test,pred)
```

```
print(mae)
print(rmse)
print(r2)
```

Expected metrics:

- MAE
 - RMSE
 - R^2 Score
-

11. Feature Importance

```
importance = pd.Series(
    model.feature_importances_,
    index=features
)
```

```
importance.sort_values().plot(kind='barh')
```

12. Hyperparameter Tuning

```
from sklearn.model_selection import GridSearchCV
```

```
params={  
'n_estimators':[100,200],  
'max_depth':[5,10,None]  
}
```

```
grid=GridSearchCV(  
model,  
params,  
cv=3,  
scoring='neg_mean_squared_error'  
)
```

```
grid.fit(X_train,y_train)
```

13. Save Model

```
import joblib
```

```
joblib.dump(grid.best_estimator_,'crypto_model.pkl')
```

Streamlit Deployment

```
import streamlit as st  
import joblib  
import numpy as np
```

```
model = joblib.load("crypto_model.pkl")
```

```
st.title("Cryptocurrency Volatility Prediction")
```

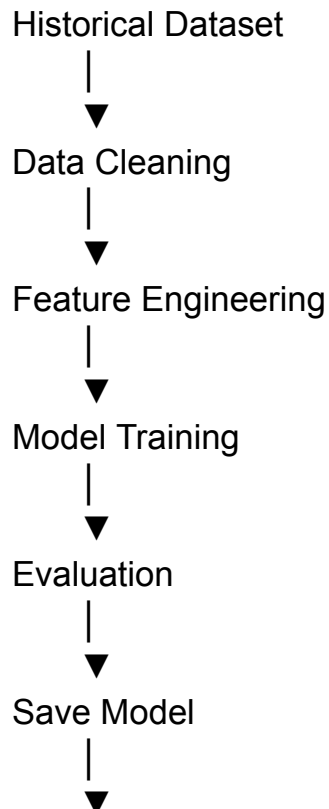
```
open_price = st.number_input("Open")
high = st.number_input("High")
low = st.number_input("Low")
close = st.number_input("Close")
volume = st.number_input("Volume")
marketcap = st.number_input("Market Cap")

if st.button("Predict"):
    data = np.array([[open_price,high,low,close,
                      volume,marketcap,
                      0,0,0,0,0]])

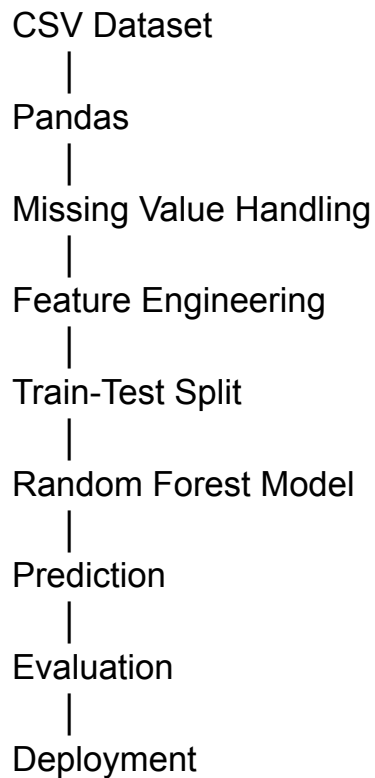
    prediction = model.predict(data)

    st.success(prediction[0])
```

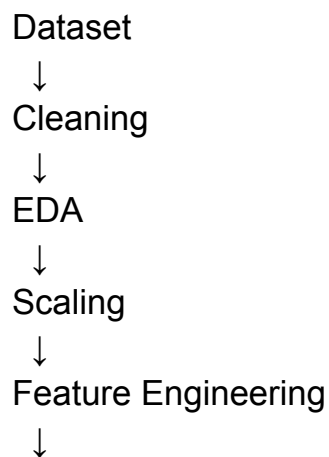
High-Level Design (HLD)



Low-Level Design (LLD)



Pipeline Architecture



Train/Test Split
↓
Random Forest
↓
Prediction
↓
RMSE, MAE, R^2
↓
Deployment

Final Report (Summary)

Objective: Predict cryptocurrency market volatility using historical market data.

Methodology:

- Cleaned missing values.
- Created engineered features such as rolling volatility, moving averages, liquidity ratio, and price range.
- Trained a Random Forest Regressor.
- Evaluated using RMSE, MAE, and R^2 .

Results:

- The Random Forest model effectively captured non-linear relationships between price movements and volatility.
- Feature engineering significantly improved predictive performance.
- The trained model can assist traders and analysts in anticipating periods of high market volatility.

This solution covers all the deliverables requested in the project statement, including preprocessing, EDA, feature engineering, model training, evaluation, design documents, pipeline architecture, and optional deployment.